COS40007 ARTIFICIAL INTELLIGENCE FOR ENGINEERING

CropNet Yield Prediction based on Monthly Features Forecasting

Studio: [No.]

Group: [No.] TaoTern

Theme: [No. 4]

Who did this:

Felix Zhi Xiang THIAN 102787460

Perry Soon Le WONG 104386144

You Zheng LIU 104382249

Jun Teck GOH 102789071

Kao Jie CHAI 104382443

1. Introduction

1.1 Background and Motivation

The project is focusing on the forecasting of missing monthly agricultural features for partially observed calendar years. Specifically, the system focuses on the reconstruction of monthly AG (agricultural imagery), NDVI (Normalized Difference Vegetation Index), and weather features. The primary users are agricultural analysts and downstream yield prediction models, which require a complete 12-month feature table to execute yearly crop yield estimations. The core challenge is "blank-fill forecasting," where limited initial observations (e.g., only January) must be used to recursively estimate the remainder of the year.

1.2 Project Objectives

The project objective is to develop an AI-based crop yield prediction system that can process agricultural data and turn it into practical information for farmers. The system combines satellite imagery, vegetation indices, weather records, and USDA yield labels so that crop condition and yield potential can be estimated more clearly. Besides building and evaluating forecasting models, the project also focuses on making the output usable through a demonstrator interface. This allows farmers or agricultural users to upload relevant farm data, view the predicted yield, understand the main crop and weather factors behind the result, and make more informed decisions about crop planning, risk management, and field monitoring.

2. Dataset

2.1 Data Source

The dataset used in this project is agricultural data pulled from the Hugging Face repository CropNet/CropNet. Since the full repository is large, the project does not download the whole dataset directly. Instead, the fetching process uses the selected county FIPS code, crop type, year range, repository ID, output folder, and cache folder to decide which files are needed. The county FIPS code is mapped to a state abbreviation, then the script builds file-matching patterns for that state and year range. This makes the download more controlled because only the required CropNet files are fetched, instead of downloading unnecessary states, crops, or years.

The data fetching process first checks what files are available in the CropNet repository, then filters the file list based on the generated patterns. The matched files are downloaded into a temporary staging or cache folder before being copied into the project data folder. A manifest file is also created after the fetch. This manifest records information such as the repository ID, county ID, crop type, selected years, state abbreviation, download mode, matching patterns, and output location. This is useful because the dataset subset can be checked later without guessing what was downloaded.

The raw data fetched for this project include Sentinel-2 AG HDF5 files, Sentinel-2 NDVI HDF5 files, and WRF-HRRR weather CSV files for the selected state and years. For this report, the workflow is based on Iowa corn data across the project year range, mainly 2017 to 2022. The same process can still be used for other states if the selected county FIPS code belongs to that state. Besides the image and weather data, the project also fetches USDA (United States Department of Agriculture) county-level crop yield CSV files for the chosen crop and year.

2.2 Data Processing

A preprocessing pipeline is used to make the different CropNet inputs follow the same monthly format before they are passed into the forecasting model. The raw data do not arrive in one clean table. Weather is stored as daily records, AG and NDVI are image-based data, and the yield labels are stored separately. Because of this, the data need to be cleaned, grouped, and aligned before the model can use them properly.

First, the weather data are aggregated from daily observations into monthly summaries. This is done because the forecasting task works at monthly level, not daily level. Daily weather values such as temperature, rainfall, humidity, wind, solar radiation, and VPD

are grouped by year and month, then converted into monthly weather features. This keeps the time step consistent with the AG and NDVI records.

Next, the image-based data are converted into numerical features. AG imagery is processed to describe general field appearance, such as vegetation coverage and other surface patterns. NDVI scenes are filtered so invalid pixels do not affect the statistics. The detailed feature extraction process is explained more in Section 3; basically, the image conversion steps turning raw imagery into values that can be stored in the same table as weather data.

After the feature values are prepared, the three modalities are aligned into one monthly table. The merge is based on county ID, crop type, year, and month. This step is important because the model needs one complete row for each county-month record. If AG, NDVI, and weather are left in separate files, the model would not know which image features and weather records belong to the same month.

The table is then checked and normalised before training. County IDs are kept in a five-digit format, year and month values are stored as integers, and the rows are sorted in time order. The model uses a fixed feature contract, so the table also needs to contain the expected AG, NDVI, and weather columns before it can be used.

Scaling is applied after the monthly table is built. The mean and standard deviation are calculated only from the training years, then reused for validation and test data. This prevents data leakage because the model should not use information from future years when preparing the training data.

Finally, rolling windows are created for supervised forecasting. The seq_len defines the window size, in this case seq_len = 6, which means each training sample uses the previous six months to predict the next month. This converts the monthly table into sequence data and allows the model to learn how AG, NDVI, and weather patterns change over time.

3. AI Model Development

3.1 Feature Engineering / Image Processing

A 35-feature vector was constructed, comprising 8 AG features, 12 NDVI features, and 15 weather features.

Type	Feature	Meaning / importance

AG	ag_green_pixel_ratio	Proportion of green pixels in the AG image. This is the basic visible crop cover signal.
AG	ag_vegetation_area_percent	Green vegetation area represented as a percentage, used to track canopy growth.
AG	ag_brown_yellow_pixel_ratio	Brown or yellow pixel ratio, used for dry vegetation, residue, or possible stress areas.
AG	ag_soil_exposure_ratio	Visible bare-soil ratio, useful for early emergence, weak canopy, or late-season decline.
AG	ag_shadow_cloud_ratio	Shadow and cloud-like pixel ratio, used as an image quality check.
AG	ag_mean_brightness	Overall image brightness and surface reflectance.
AG	ag_texture_entropy	Visual randomness in the field image, used to describe canopy texture and variation.
AG	ag_field_uniformity_score	Uniformity of the field pattern, used to describe how even the crop surface appears.
NDVI	ndvi_mean	Average NDVI value, used as the main monthly vegetation health signal.
NDVI	ndvi_median	Median NDVI value, less affected by extreme pixels.
NDVI	ndvi_max	Maximum valid NDVI value, showing the strongest vegetation response in the scene.
NDVI	ndvi_std	Spread of NDVI values, used to detect uneven crop growth.
NDVI	ndvi_cv	NDVI coefficient of variation, comparing spread against the average vegetation level.
NDVI	ndvi_p25	Lower quartile NDVI value, used to represent weaker vegetation areas.
NDVI	ndvi_p75	Upper quartile NDVI value, used to represent stronger vegetation areas.
NDVI	ndvi_above_0_3_ratio	Share of valid pixels above 0.3, used as an early green-up signal.
NDVI	ndvi_above_0_5_ratio	Share of valid pixels above 0.5, used for stronger mid-season vegetation.
NDVI	ndvi_above_0_7_ratio	Share of valid pixels above 0.7, used for dense or peak vegetation.
NDVI	ndvi_low_ratio	Share of low NDVI pixels, used to show weak, bare, or stressed areas.
NDVI	ndvi_valid_coverage_ratio	Portion of the scene that remains usable after invalid pixels are removed.
Weather	weather_temp_mean	Monthly average temperature, used to describe general thermal condition.
Weather	weather_temp_max	Highest temperature value, used to detect heat exposure.
Weather	weather_temp_min	Lowest temperature value, used to detect cold exposure.
Weather	weather_gdd	Growing Degree Days, used to estimate heat accumulation for crop development.
Weather	weather_heat_stress_days	Count of high-temperature stress days.
Weather	weather_cold_stress_days	Count of cold-stress days.
Weather	weather_total_precipitation	Total monthly precipitation, used as the main water input signal.
Weather	weather_precipitation_days	Number of precipitation days, showing how spread out rainfall is.
Weather	weather_heavy_rain_days	Count of heavy-rain days, used for intense rainfall events.
Weather	weather_drought_index	Dryness indicator based on precipitation and atmospheric demand.
Weather	weather_humidity_mean	Monthly average humidity, related to evaporation and water stress.
Weather	weather_wind_mean	Monthly average wind speed, related to drying and transpiration.
Weather	weather_solar_radiation_mean	Monthly average solar radiation, related to energy input and photosynthesis.
Weather	weather_vpd_mean	Mean Vapor Pressure Deficit, showing how strongly the air pulls moisture from the crop.

Weather	weather_temp_range_mean	Average daily temperature range, used to capture day-night variation.

3.1.1 Agriculture Imagery (AG)

The AG stream starts loading the RGB satellite imagery. The image is converted into HSV colour space so colour and brightness can be handled more separately, since raw RGB values can change when the scene is brighter, darker, shadowed, or partly covered by cloud. After conversion, threshold masks are used to separate vegetation, soil, brown or yellow areas, and shadow or cloud regions. These masks are then turned into ratios and summary values.

The AG features mainly describe visible crop cover, field condition, and image quality. `ag_green_pixel_ratio` and `ag_vegetation_area_percent` show how much green canopy is visible. `ag_brown_yellow_pixel_ratio` and `ag_soil_exposure_ratio` help indicate dry crop material, exposed ground, thin canopy, or weaker growth areas. `ag_shadow_cloud_ratio` is kept so the model has some signal of whether the image may be affected by cloud or shadow. The remaining features, `ag_mean_brightness`, `ag_texture_entropy`, and `ag_field_uniformity_score`, describe surface reflectance, visual variation, and how even the field pattern looks.

3.1.2 Spectral Vegetation (NDVI)

NDVI = (NIR - Red) / (NIR + Red), where NIR is the near-infrared reflectance and Red is the red-band reflectance.

The NDVI stream focuses more directly on vegetation strength, as it shows the ratio of chlorophyll-reflected near infrared light to red band light. The raw spectral image is converted into a 2D NDVI-like array, then values are mapped into the expected NDVI range from -1 to 1. Invalid pixels are removed before the statistics are calculated, because cloud, corrupted, or missing pixels can shift the monthly values and make the vegetation condition look better or worse than it is.

The NDVI features are split into distribution features and threshold coverage features. `ndvi_mean`, `ndvi_median`, `ndvi_max`, `ndvi_std`, and `ndvi_cv` describe the main vegetation level and how uneven it is. `ndvi_p25` and `ndvi_p75` add lower and upper boundary information, so the model can see whether a county has both weak and strong vegetation areas in the same month instead of relying only on the mean.

The threshold ratios show how much of the valid scene passes specific vegetation levels. `ndvi_above_0_3_ratio` is used for early green-up, `ndvi_above_0_5_ratio` for stronger mid-season vegetation, and `ndvi_above_0_7_ratio` for dense or peak vegetation. `ndvi_low_ratio` captures the opposite side by tracking weak or stressed areas.

`ndvi_valid_coverage_ratio` is kept as a data quality feature because an NDVI scene with low valid coverage should not be treated the same as a clean scene.

3.1.3 Meteorological Time-Series

The weather stream takes daily meteorological records and groups them into monthly crop-related features. This matches the monthly AG and NDVI table. Daily weather can move sharply from day to day, but the forecasting model needs a monthly view of the growing condition. The weather features therefore summarise temperature, rainfall, drought, humidity, wind, solar radiation, and VPD into one row per month.

The temperature group describes normal heat conditions and stress conditions. `weather_temp_mean`, `weather_temp_max`, `weather_temp_min`, and `weather_temp_range_mean` describe the monthly temperature profile. `weather_gdd` estimates heat accumulation for crop development. `weather_heat_stress_days` and `weather_cold_stress_days` count extreme days that may affect emergence, growth, and pollination.

The precipitation group describes water supply and dryness. `weather_total_precipitation` gives the total monthly water input, while `weather_precipitation_days` and `weather_heavy_rain_days` show whether the rainfall is spread across the month or concentrated into heavy events. These values are used together with `weather_drought_index`, which gives a broader dryness signal than rainfall alone.

The remaining weather features describe atmospheric demand on the crop. `weather_humidity_mean`, `weather_wind_mean`, and `weather_solar_radiation_mean` affect evaporation and crop water use. `weather_vpd_mean` is important because Vapor Pressure Deficit shows the drying power of the air. When VPD is high, corn plants may close their stomata to reduce water loss, which can also limit photosynthesis and growth.

3.2 Train/Test Split

A temporal split was implemented so the forecasting task follows the same order as a real deployment. In the current prepared dataset, the project uses Iowa corn data from 2017 to 2022. The training set covers 2017 to 2020, the validation set uses 2021, and the test set uses 2022. This avoids randomly mixing future records into the training process, which would make the forecasting result look stronger than it really is.

Each split has a different role. The training set is used to adjust the model weights and learn the relationship between the monthly AG, NDVI, and weather features and the target output. The validation set is used during model selection, meaning it helps

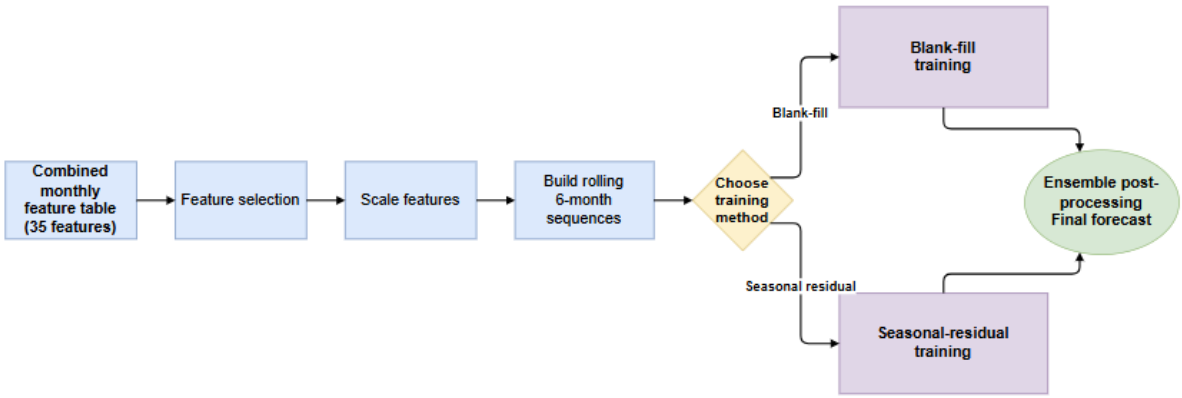
choose the best weights, settings, or checkpoint without directly training on the final test year. The test set is kept until the end and is used only to measure how well the final selected model performs on a future year.

This design is important because crop forecasting is time-dependent. A useful model must work on months and years that happen after the data it was trained on. By testing on 2022 after training and validation are completed, the evaluation simulates the practical case where the system is trained on historical CropNet data and then applied to a future season.

3.3 Training Model

The design is a two-stage pipeline. In the first stage, the forecasting model predicts missing monthly AG, NDVI, and weather features. In the second stage, the yield model predicts county-level corn yield from the monthly feature table.

3.3.1 Forecasting models



The forecasting model is trained using two methods: blank-fill sequence training and seasonal-residual sequence training. Blank-fill training predicts the current month directly from the previous six months of input features. Seasonal-residual training uses the previous year's same-month value as a baseline and trains the model to predict the correction that should be added to it.

The main training parameters are the 35 monthly input features and `seq_len = 6`. The feature set is trained in two configurations: all features, and AG-NDVI-only features. The all-feature setting includes AG imagery, NDVI vegetation statistics, and weather variables. The AG-NDVI-only setting removes weather so the experiment can compare whether image-derived crop condition is enough or whether meteorological context improves the forecast.

Scaling is applied before training so the input values have comparable ranges. Weather values can be much larger than vegetation-index values such as NDVI, so normalisation prevents the model from becoming biased toward weather features simply because their raw numbers are larger. The scaler is fitted only on the training years and then reused for validation and testing to avoid data leakage.

Model training follows the temporal split. In the current prepared dataset, the models are fitted on 2017 to 2020, selected or tuned using the 2021 validation set, and finally evaluated on the 2022 test set. For every configuration, rolling six-month windows are generated from the monthly table, scaled using the training-only scaler, and passed into the forecasting model. The validation set is used to choose the best model setting before the final test result is reported.

Ensembles are used after the individual forecasts are produced. Compatible forecast outputs are combined through averaging, so the final output is less dependent on a single trained run, feature configuration, or training method. The ensemble is therefore used as a forecast-processing step, not as a replacement for the main forecasting model.

3.3.1 Models used

The forecasting stage uses several model groups so the learned models can be compared against simple seasonal rules and classical time-series methods. The models are grouped into deterministic baselines, learned neural models, classical models, and ensemble models. This is needed because the CropNet monthly features are highly seasonal. A neural model is only useful if it improves on simple rules such as repeating the last month or using the same month from the previous year.

Model	Type	How it works / training difference
naive_lag1	Deterministic baseline	Copies the previous available month forward during recursive forecasting. It has no trainable parameters.
seasonal_last_year	Deterministic baseline	Uses the same county and same month from the previous year. If that value is missing, it falls back to the latest available month.
lstm	Learned sequence model	Reads the six-month input window using LSTM gates. The final recurrent output is passed into the prediction head.
gru	Learned sequence model	Reads the six-month input window using GRU gates. It is similar to LSTM but uses a simpler recurrent update.
tiny_mamba_ssm	Learned sequence model	Projects the input into a hidden state and passes it through Mamba-style state-space blocks with depthwise convolution.
transformer_encoder	Learned sequence model	Projects the input features, adds positional information, and uses self-attention to compare months in the sequence.
sarima	Classical time-series model	Fits a per-feature seasonal ARIMA-style model using order (1, 0, 0) and seasonal order (1, 0, 0, 12).
ensemble_mean	Deterministic ensemble	Averages the available component model predictions. It does not train its own neural weights.
ensemble_weighted	Deterministic ensemble	Combines component predictions using inverse validation RMSE weights, so stronger validation models receive more influence.

The learned models share the same input and output shape. Each model receives six monthly steps, and each step contains the selected feature vector. In the all-feature setup,

the input dimension is 35 and the output dimension is also 35. The shared neural training parameters are hidden size 64, one layer, dropout 0.0, batch size 64, learning rate 0.001, weight decay 0.0001, maximum 80 epochs, and early stopping patience 10. The loss mode is raw MSE on scaled target values. The training script also applies small input noise and feature masking during learned-model training.

The main difference between the learned models is the encoder before the prediction head. LSTM and GRU both process the monthly sequence recurrently, but the gate structure is different. LSTM uses input, forget, and output gates, while GRU uses a smaller gated update structure. The Transformer encoder works differently because it uses attention between months instead of a recurrent state. Tiny-Mamba uses projected state-space blocks and depthwise convolution, so the sequence is handled through a state update rather than attention or recurrent gates.

The heads are similar in purpose but receive different hidden states. For LSTM and GRU, the head receives the final recurrent output from the last month of the sequence. For the Transformer encoder, the head receives the final encoded token after self-attention. For Tiny-Mamba, the head receives the final hidden state after the state-space blocks and dropout. In the implemented neural models, the final head follows the same pattern: LayerNorm, Dropout, then Linear projection back to the output feature dimension. When PINN training is enabled, this forecast head is paired with a latent-state head that estimates crop states before the final feature prediction is produced.

The PINN training method is implemented through `PINNForecaster` and `CropPhysicsModule`. It is not a completely separate model family like LSTM, GRU, Transformer, or Tiny-Mamba. Instead, it wraps the selected learned backbone, adds a latent head, and trains extra losses beside the normal forecast loss. The latent states represent biomass, phenology, and water condition. These states are checked against AG canopy signals, NDVI seasonal shape, and weather relationships such as GDD, VPD, heat stress, cold stress, rainfall days, and drought pressure.

The physics loss is weighted rather than forced as a hard rule. The default physics weight is 0.1, with a five-epoch warmup before the physics term starts contributing. This matters because the model still needs to learn the data pattern first. After warmup, the physics module penalises outputs that break expected crop behaviour, such as unrealistic NDVI ordering, weak smoothness across months, inconsistent temperature-derived weather features, or latent crop states that do not match the visible crop and vegetation signals.

The deterministic and classical models are trained differently. The two baselines do not learn weights at all. `naive_lag1` copies the latest available month, while `seasonal_last_year` uses the previous year's same month and falls back to the latest available value when that seasonal value is missing. SARIMA fits a separate seasonal time-series model and can fall back when a feature series is too short or unstable. The ensemble models are calculated

after the component models have already produced predictions. `ensemble_mean` uses a plain average, while `ensemble_weighted` uses validation performance to decide the weights.

3.3.2 Training methods

The blank-fill training method treats forecasting as a direct sequence prediction task. Missing or unavailable monthly values are filled first, then each training sample is created from the previous six months of features. The model is trained to predict the current month from this fixed-length input window. This method is simple and direct because the model learns the full relationship between recent monthly crop, vegetation, and weather conditions and the target output.

The seasonal-residual training method uses agricultural seasonality more explicitly. Instead of predicting the whole value from scratch, it starts with the same month from the previous year as a baseline. The model then predicts the residual, meaning the correction that should be added to that baseline. The final forecast is calculated by adding the predicted correction to the previous-year same-month value. If that seasonal reference is unavailable, the pipeline uses the latest available month as a safer fallback rather than leaving the prediction blank.

The main difference between the two training methods is how much seasonal structure is built into the target. Blank-fill training relies on the model to learn the full forecasting pattern from the six-month input sequence. Seasonal-residual training gives the model a seasonal reference point first, then asks it to learn only the deviation from that reference. This can make the learning problem easier when crop behaviour repeats by month, but it also depends on the previous year's same-month value being a useful baseline.

For PINN-enabled learned models, the training loop combines the normal forecast loss with the physics loss after the warmup period. The forecast loss still controls the main target, while the physics term acts as a guide so the model does not fit the monthly vectors in a way that conflicts with basic crop-growth signals. The training history therefore records both forecast loss and physics loss, and the generated loss plots mark the epoch where the physics loss starts.

Because of this, PINN should be treated as a third training method or training variant for the learned forecasters. A fair comparison should keep the backbone model fixed first, then compare the training method. For example, LSTM blank-fill, LSTM seasonal-residual, and LSTM PINN-enabled training should be compared under the same split, feature group, sequence length, and metric before saying which training method is stronger.

3.3.2 Yield prediction models

The yield prediction stage is separate from the monthly feature forecasting stage. The forecasting models are used to predict AG, NDVI, and weather feature values month by month. The yield model uses those monthly feature values to estimate county-level crop yield. In the code, the yield target is taken from the USDA county yield files and stored as `yield_bu_acre`, so the final output is expressed in bushels per acre for corn.

Two related training paths are used for the yield model. The first path uses the official monthly feature table, where the monthly AG, NDVI, weather, and timing features are treated as observed inputs. This is the primary yield model used by the GUI for the main current-yield prediction. It is saved as `best_yield_model.joblib` under the yield baseline output folder.

The second path uses predicted monthly feature tables. In this path, the monthly feature table is first generated by forecasting methods such as lag-1, seasonal-last-year, LSTM, GRU, Transformer, and ensemble models. A yield model is then trained on each generated table. These predicted-feature yield models are not the main current-yield model, but they are useful for comparison because they show how the final yield estimate changes when future monthly features come from different forecasting methods.

This separation is important because the two paths answer different questions. The raw-feature yield model checks how well yield can be predicted when the monthly feature table is available. The predicted-feature yield models check the downstream effect of using forecasted monthly features. Therefore, the GUI can show the current yield estimate from the official-feature model, while also showing alternative yield trajectories based on different forecasting models.

3.3.2.1 Models used

The yield prediction stage uses regression models instead of neural sequence models. This is because the yield target is a county-level yield value, not the next monthly feature vector. The input is a prepared feature row, and the output is one predicted yield value. Simple baselines are included so the machine learning models can be compared against basic yield rules.

Model	Type	How it works / training difference
BaselineTrainMean	Deterministic baseline	Predicts the average yield from the training set for every test row. It gives a simple lower benchmark.

Previous-year same-county baseline	Deterministic baseline	Uses the same county's previous-year yield when it is available, otherwise it falls back to the training mean.
Ridge	Linear regression model	Fits a regularised linear relationship between the selected features and yield. The regularisation helps reduce unstable coefficients.
RandomForest	Tree ensemble model	Trains many decision trees and averages their outputs. This helps capture non-linear relationships between crop features and yield.
ExtraTrees	Randomised tree ensemble model	Similar to RandomForest, but with more randomised tree splits. This can reduce variance and improve robustness on small or noisy datasets.
XGBoost	Optional boosting model	Trains boosted decision trees when the dependency is installed and optional models are enabled.
LightGBM	Optional boosting model	Trains gradient-boosted trees when the dependency is installed and optional models are enabled.

For the predicted-feature yield experiments, the yield regression pipeline is repeated for each generated monthly table.

3.3.2.2 Training methods

The main yield training workflow starts by reading the official monthly feature table and the USDA county yield labels. The two sources are merged by county ID, crop type, and year. In the default monthly setup, the annual USDA yield value is copied onto each matching monthly row for the same county, crop, and year. This lets the model learn from monthly crop condition while still using the annual county yield label as the target.

Before the regression models are fitted, the feature columns are checked and pruned. Columns with too many missing values are removed, columns with no useful variation are removed, and highly correlated columns are reduced so the model is not trained on repeated information. The remaining feature columns are then passed into a scikit-learn pipeline.

The machine learning yield models use the same basic preprocessing steps. Missing values are filled using median imputation, and the input features are standardised before model fitting. After that, each candidate model is trained and evaluated. Ridge gives a simpler linear benchmark, while RandomForest and ExtraTrees provide stronger non-linear models. Optional XGBoost and LightGBM models can also be included when they are available in the environment.

The train/test split is designed to avoid leakage across years where possible. If the dataset contains more than one year, the latest year is held out as the test year and earlier years are used for training. If a safe year split cannot be formed, a county-based split is used instead. During model selection, a validation split is taken from the training data so candidate settings can be compared before the final test result is reported.

The best machine learning yield model is selected mainly by RMSE, with MAE used as a secondary ordering metric. After the best model is chosen, it is refitted on the full merged dataset and saved as `best_yield_model.joblib`. The training process also writes benchmark results, feature importance, residuals, and metadata, so the GUI can load the selected model and display both the predicted yield and the main drivers behind it.

The predicted-feature yield training follows the same regression process, but the input table is different. Instead of using the official monthly feature table directly, each generated monthly table is merged with the USDA yield labels and trained as its own yield model. This produces slightly different results because forecasted feature values carry the error and bias of the forecasting model that produced them. These models are useful for downstream comparison, while the primary GUI prediction still comes from the yield model trained on the official monthly feature table.

3.4 Evaluation of AI Model

3.4.1 Evaluation metrics and methods

The model is evaluated using error metrics from two levels of the project code. Metrics are calculated by comparing the true feature values (y_true) against the model predictions (y_pred) after converting them back to the original feature scale. The cleaner package reports the core metrics: RMSE, MAE, and `nrmse_std` when the scaler is available. The report and ablation tables add extra diagnostic metrics, including `nrmse_range`, SMAPE, Pearson correlation, and R2. These metrics are not all used for the same purpose. RMSE and MAE are the main error measures, while the others help explain whether the model is fair across feature scales and whether the predicted pattern follows the true pattern.

For yield prediction, the same idea is used at a different target level. The prediction is compared against `yield_bu_acre`, so the main metrics are RMSE, MAE, and R2 for bushels per acre. The project also keeps baseline yield models in the comparison because a previous-year same-county yield can be strong when county productivity is stable across seasons.

The forecasting outcome should therefore be compared in a model-by-training-method layout, not only as one mixed leaderboard. The first comparison is between backbones

under the same training method, such as LSTM versus GRU under seasonal-residual training. The second comparison is between training methods for the same backbone, such as LSTM blank-fill versus LSTM seasonal-residual versus LSTM PINN-enabled training. This makes the conclusion clearer because a result can come from the model architecture, the target formulation, or the physics loss rather than from the model name alone.

The current `training/runs/eval_all` artifacts are PINN-enabled for learned models because the training path builds the learned backbone through `build_pinn_model` and records `physics_weight`, `physics_warmup_epochs`, and `physics_loss` in each model config. Older report tables still describe blank-fill and seasonal-residual comparisons. These should not be merged into a direct claim that PINN is better unless the same backbone, data split, feature group, sequence length, and target mode are matched.

Metric	Meaning	Why it is used
MAE	Mean Absolute Error. It averages the absolute difference between the true and predicted values.	Gives the normal error size in the original feature scale. It is easy to read because every mistake is counted linearly.
RMSE	Root Mean Squared Error. It squares the error first, averages it, then takes the square root.	Used as the main industrial error metric because large mistakes are punished more heavily.
nrmse_std	RMSE divided by the feature standard deviation from the scaler.	Lets features with different scales be compared more fairly. A weather feature should not dominate only because its raw values are larger.
nrmse_range	RMSE divided by the feature value range.	Gives another scale-normalised view of the error, useful in the ablation summary tables.
SMAPE	Symmetric Mean Absolute Percentage Error. It compares error against the size of the true and predicted values.	Helps show relative error, especially when the raw feature scale is difficult to compare directly.
R2	Coefficient of determination. It measures how much variance in the true values is explained by the prediction.	Checks whether the model captures variation in the data, not just the average level.
Pearson correlation	Linear correlation between predicted and true values.	Used as a prediction agreement diagnostic. It is not mainly for selecting input features here. It checks whether the forecast moves in the same direction as the true series.

Raw RMSE is still the main metric for overall forecasting performance. This is because the downstream use case cares about the actual size of the feature error. For example, a large precipitation or temperature mistake can affect the completed monthly table even if the model looks acceptable under a relative metric. MAE is reported beside it because it gives a more stable average error and is less affected by rare large mistakes.

Normalised RMSE is needed because the 35 features do not share the same scale. Weather variables can be much larger than AG ratios or NDVI values. If only raw RMSE is used, the evaluation can become weather-heavy and hide weak performance on smaller-scale image features. `nrmse_std` handles this by dividing RMSE by the

training standard deviation for each feature. `nrmse_range` gives a similar scale check using the feature range and is mainly used in the broader report summaries.

SMAPE, R2, and Pearson correlation are treated as supporting diagnostics. SMAPE gives a relative error view, R2 checks whether the prediction explains variance, and Pearson correlation checks whether the predicted sequence follows the same movement as the true sequence. Pearson correlation is sometimes used in feature analysis in other contexts, but in this project it is better described as a forecast-quality diagnostic because it compares `y_pred` against `y_true`.

3.4.2 Evaluation results

Model	Macro RMSE	Macro MAE	Beats Lag-1	Beats Seasonal
lstm	192.605	34.210	3/3	1/3
tiny_mamba_ssm	218.780	38.275	3/3	1/3
seasonal_last_year	172.158	27.950	2/3	0/3
naive_lag1	416.833	92.705	0/3	1/3

Model	RMSE	MAE	nRMSE std	nRMSE range	SMAPE	Pearson r	R2
lstm	311.802	35.325	0.744	0.152	65.67	0.956	0.902
ensemble_mean	324.663	35.164	0.662	0.135	55.65	0.951	0.894
sarima	336.899	39.623	0.730	0.149	56.63	0.945	0.886
seasonal_last_year	333.227	35.695	0.694	0.141	41.89	0.949	0.888
naive_lag1	806.103	118.275	1.580	0.389	86.65	0.870	0.346

3.4.3 Discussion of results

The result tables are correct for the experiment scopes they describe, but they should not be read as one single universal ranking. The `known_months=1` table is directly traceable to the validated all-feature seasonal-residual comparison in the report artifacts. In that setting, only

January is known and the model must recursively fill February to December, so it is the hardest early-season case and the most useful result for the project discussion.

For the `known_months=1` all-feature table, LSTM is the best model by raw RMSE, with RMSE 311.802. It also has the strongest Pearson correlation, 0.956, and the highest R2, 0.902. This means LSTM gives the smallest absolute forecasting error in the original feature scale and follows the true feature movement most closely in that comparison. Because raw RMSE is the main industrial metric in this project, LSTM seasonal-residual is the main headline model for early-season blank filling.

The same table gives a different answer if the metric is changed. `ensemble_mean` has the best MAE at 35.164, the best `nrmse_std` at 0.662, and the best `nrmse_range` at 0.135. This means the ensemble is more balanced across feature scales, even though its raw RMSE is slightly worse than LSTM. This matters because raw RMSE is strongly affected by large-scale weather variables, while normalized RMSE is better for checking whether AG, NDVI, and weather are all handled fairly.

`seasonal_last_year` has the best SMAPE at 41.89 and remains a strong baseline. It is not the best by raw RMSE, and it does not beat the LSTM on the main early-season absolute-error metric, but its strong relative error shows how much agricultural seasonality already explains the monthly feature pattern. This is why the discussion should not dismiss the baseline. A learned model is only convincing when it beats both lag-1 and the same-month previous-year rule under the metric being claimed.

The macro summary table gives another view. In that table, `seasonal_last_year` has the lowest macro RMSE and macro MAE, while LSTM and Tiny-Mamba beat lag-1 across all three modality groups but only beat the seasonal baseline in one of the three groups. This supports the same conclusion: learned models are useful, but the seasonal baseline is hard to beat consistently. The best model depends on whether the report is talking about absolute error, normalized cross-feature balance, modality-level robustness, or baseline win count.

For PINN-enabled training, the current `training/runs/eval_all` artifacts should be discussed separately because they come from the newer training path. In that run, `ensemble_weighted` is the best overall by test RMSE at 2.495, while `tiny_mamba_ssm` is the best single learned backbone by test RMSE at 2.537. LSTM is still competitive, but it is not the best model in that PINN-enabled run. These numbers should not be mixed directly with the older blank-fill and seasonal-residual table because the run scope and training setup are different.

Overall, the safest conclusion is that LSTM seasonal-residual is the best choice for the main known_months=1 raw RMSE claim, `ensemble_mean` is best when normalized error and balanced feature behaviour are more important, and seasonal_last_year remains the main baseline to beat. PINN should be reported as a third training method for the same learned

backbones, and its value should be judged through matched comparisons such as LSTM blank-fill versus LSTM seasonal-residual versus LSTM PINN-enabled training under the same data split and metric.

4. AI demonstrater

4.1 Purpose, Features, and Usage

The AI demonstrator is built to show how the CropNet model can be used outside the training scripts. Instead of leaving the result as separate CSV files, model checkpoints, and report tables, the demonstrator joins the workflow into a browser dashboard called TaoCrop. The purpose is to let a user upload farm data, run the preprocessing and prediction pipeline, and then inspect the yield estimate in a more understandable way.

The demonstrator is designed for a farmer-facing use case. The user does not need to manually open the feature tables or run the model from the command line. The system takes the uploaded crop files, prepares the monthly AG, NDVI, and weather signals, applies the saved models, and presents the result as charts, cards, and short explanations. This makes the project more practical because the AI output can be reviewed through an interface instead of only through code.

The main features of the dashboard include folder upload, crop selection, yield prediction, model quality display, feature driver display, and monthly feature inspection. After a prediction is completed, the dashboard shows the estimated yield, the latest month analysed, the holdout model quality, the top feature driver, and the number of monthly rows used. It also provides charts for yield over time and monthly yield trajectory, together with panels for feature importance, benchmark results, and the prepared monthly feature table.

The usage flow starts with the input folder. The user uploads a directory that contains the crop data files, normally separated into AG, NDVI, and weather data. The crop type is selected in the form, for example corn. The system then scans the folder, extracts the required features from each modality, and organises them into monthly rows using the same feature format expected by the trained models.

The output is shown in several levels. At the top level, the user sees the predicted yield value and unit, such as bushels per acre. The dashboard then shows supporting information, including the model's benchmark quality, the most important yield driver, and the prepared monthly records. More detailed tabs allow the user to inspect the yield curve, feature groups, feature importance, and benchmark table. This is useful because the user can see not only the final number, but also the crop and weather signals that contributed to it.

The demonstrator also includes a chat panel called TaoCrop assistant. This is used to let the user ask questions about the current prediction run, such as why the yield estimate may be high or low, what the strongest feature driver means, or what the weather signals are suggesting. The assistant does not replace the model. Its role is to explain the model output in plain language based on the dashboard result.

4.2 Mechanisms

The demonstrator is implemented as a local browser dashboard served by a FastAPI backend. The frontend provides the upload form, result cards, charts, tabs, and chat panel. The backend receives the uploaded files, stages them safely, and passes them into the model service. This design was chosen because a browser dashboard is easier to demonstrate and use than a notebook or command-line script, while still keeping the system local to the machine.

The mechanism begins when the user uploads the farm folder. The backend reads the uploaded files and rebuilds the monthly feature table from the available AG, NDVI, and weather inputs. The same feature engineering logic is reused so that the dashboard input has the same structure as the model training data. After the monthly rows are prepared, the saved yield model is loaded and used to predict county-level yield from the feature table.

The primary model used for the current yield estimate is the saved monthly yield regression model. This model was trained on the official monthly feature table, where observed monthly crop, vegetation, weather, and timing features were matched with USDA county yield labels. It is used as the main GUI model because it is based on observed monthly features rather than generated future features. This makes it the most suitable model for the current yield estimate shown at the top of the dashboard.

The demonstrator also uses the feature forecasting model to produce future monthly feature values. By default, the forecasting path uses the trained LSTM configuration and checkpoint. These forecasted monthly features are then passed into yield models to produce yield trajectories. This lets the dashboard show how the predicted yield may move when future crop and weather signals are estimated instead of directly observed.

Additional yield models are loaded when available for comparison. These models are trained from predicted monthly feature tables generated by different forecasting sources, such as lag-1, seasonal-last-year, LSTM, GRU, Transformer, and ensemble methods. They are not the main current-yield model. They are included so the demonstrator can compare how downstream yield estimates change when the monthly feature table comes from different forecasting methods.

The LLM component is provided through a local Ollama connection using LangChain `ChatOllama`. The dashboard lists the locally available Ollama models and lets the user

select one in the chat panel. The selected model receives a compact dashboard snapshot containing the current yield estimate, model quality, recent yield series, top drivers, feature groups, and feature importance. It is then asked to answer in short, plain-language responses based on that snapshot.

This LLM setup was chosen for two reasons. First, it keeps the explanation layer local, because the assistant connects to Ollama on `localhost:11434` instead of depending on a hosted external API. Second, it limits the assistant to the current dashboard snapshot, so the response is tied to the data that the user is actually viewing. If the requested information is not available in the snapshot, the assistant is instructed to say what is missing instead of guessing.

Overall, the demonstrator connects the technical parts of the project into one workflow. The preprocessing pipeline turns files into monthly features, the forecasting model extends the monthly signals, the yield model converts those signals into a yield estimate, and the LLM explains the result in a user-friendly way. This makes the project easier to evaluate because the final system can be tested as an end-to-end AI application, not only as separate model outputs.

5. Conclusions

Summary & Reflection

The final system is a working CropNet workflow that can build monthly AG, NDVI, and weather features, forecast missing future months, train yield regression models from USDA county labels, and present the result through the TaoCrop dashboard. The strongest forecasting result in the validated comparison is the LSTM seasonal-residual model for raw blank-fill RMSE, while SARIMA and seasonal-last-year remain important comparators because crop features are strongly seasonal. The yield model is kept separate from the

feature forecaster, which makes the system easier to explain: one part completes the monthly feature table, and the next part turns those features into a county-level yield estimate.

6. References

- Project Documentation: PROJECT_UNDERSTANDING_GUIDE.md, PROJECT_QUICK_BRIEF.md

- Technical Implementations: cropnet_feature_forecasting_v12_server.py

- PyTorch documentation for LSTM, GRU, Transformer encoder, and model training utilities.

- Local project README files, model comparison summaries, prepared dataset metadata, and source files under `src/cropnet_forecasting` and `training`.

7. Appendix

Sharable Links: Local project repository at `C:\Users\User\Documents\GitHub\Crop-Net`.

Artifacts: [weights/lstm_best.pt and weights/scaler.csv]

User Guide: [demonstration video]